Creating and Integrating a CPU Usage KPI in O-RAN Systems

Professors : **Serge FDIDA, Mai Trang NGUYEN**

Lab Assistant : **Taha Mohsen Zaidi**

Etudiants : **Emilie LIN, Eric ZHOU, Paul XU, Ilyas Benyahia**

Table des matières

1 Repository

All code modifications are located in the 5g-workspace subfolder of the following repository : https://github.com/DHinode/cell_oran_kpi

2 Introduction

O-RAN enables open, flexible, and intelligent control of Radio Access Networks through standardized interfaces and the RAN Intelligent Controller (RIC). By decoupling hardware and software components and exposing RAN functionalities through open interfaces, O-RAN facilitates advanced monitoring and optimization via software-based applications. While existing Key Performance Measurements (KPMs) mainly focus on radio and traffic-related indicators, visibility into the computational resources of RAN nodes remains limited. As modern 5G RAN implementations increasingly rely on software processing running on general-purpose hardware, the computational load of RAN nodes becomes a critical factor affecting latency, throughput, and overall system stability. In particular, CPU overload at the gNB can degrade radio performance and limit the effectiveness of real-time control applications, highlighting the need for computational resource awareness within the O-RAN framework.
This project extends the FlexRIC KPM Service Model by introducing a CPU Usage Key Performance Metric for O-RAN nodes. The proposed metric enables real-time monitoring of CPU utilization via the E2 interface and allows xApps to subscribe to and process computational performance data. The implementation includes modifications to the KPM Service Model, updates to E2 nodes, and the development of an xApp for metric consumption.
The system is deployed using a hybrid setup with a containerized 5G Core Network and bare-metal RAN and FlexRIC components, providing a realistic and low-latency environment for testing and validating the proposed CPU usage metric.

3 Objectives

1. **Create a Custom Metric :**
   Define and implement a new Key Performance Metric (KPM) to monitor the CPU utilization of O-RAN nodes, using standard system monitoring tools / APIs.

2. **Extend the KPM Model :**
   Modify the existing KPM service model in FlexRIC to incorporate the new metric, ensuring compatibility and integration.

3. **Update E2 Nodes :**
   Configure E2 nodes to support and report the newly created metric to the FlexRIC

controller.

4. **Develop an xApp :**
   Build and deploy an xApp that subscribes to the new metric, processes the data, and takes actions based on the analysis.

5. **Test and Validate :**
   Deploy the enhanced system in a FlexRIC environment to test and verify the accuracy and reliability of the new metric.

4 Tools & Software

- **Docker-Compose :** For deploying containerized 5G Core Network (CN) components.
- **OpenAirInterface (OAI) :** For implementing and testing the 5G RAN.
- **FlexRIC :** For extending the KPM service model and developing the xApp.
- **Bare-Metal Deployment :** For deploying the RAN and FlexRIC controller to ensure high performance in real-world scenarios.

5 Technical Background

5.1 FlexRIC and OpenAirInterface

FlexRIC is an open-source implementation of the O-RAN Near-RT RIC designed for low-latency and resource-efficient operation. Its modular architecture implements Service Models as shared dynamic libraries, enabling flexible extension and minimizing coupling between components. FlexRIC also introduces the E42 interface, which enhances communication between the Near-RT RIC and xApps and simplifies subscription and control procedures.
OpenAirInterface (OAI) is an open-source platform implementing 4G and 5G RAN components. In this project, OAI is used to implement the 5G gNB with an integrated E2 Agent, allowing the gNB to function as an E2 node capable of reporting KPM measurements to the Near-RT RIC.
The combination of FlexRIC and OAI provides an open and realistic O-RAN-compliant environment for prototyping and validating extensions to the KPM Service Model. This project leverages this environment to introduce a CPU usage KPM, enabling visibility into the computational state of the gNB and supporting CPU-aware monitoring through xApps.

5.2 xApps

xApps are software applications running on the Near-RT RIC that monitor and control RAN functions using data exposed through the E2 interface. They operate on time scales between approximately 10 ms and 1 s and enable data-driven optimization of RAN performance through standardized Service Models.

5.3 E2 Interface and Service Models

The E2 interface is a key open interface connecting the Near-RT RIC with RAN nodes, such as gNBs, CUs, and DUs. It enables both the collection of performance measurements and the execution of control actions. Each RAN node exposes its capabilities through an E2 Agent, which handles E2 signaling and communication with the RIC using the E2 Application Protocol (E2AP).
RAN functionalities are exposed via Service Models (E2SMs), which define the data structures and procedures used for monitoring and control. The Key Performance Measurement (KPM) Service Model allows xApps to subscribe to performance metrics that are periodically or conditionally reported by E2 nodes.

6 Project setup

Before going further with the project, we need to set up the 5G Core Network and launch the gNB, the UE, and the FlexRIC controller. Then, we need to open several terminals to ping each end of the network in order to test reachability.

- Build OAI with E2 Agent

```
$ Build OAI

cd ~/cell_oran_kpi/5g-workspace/oai/cmake_targets
sudo ./build_oai -I -w SIMU --gNB --nrUE --build-e2 --ninja
```

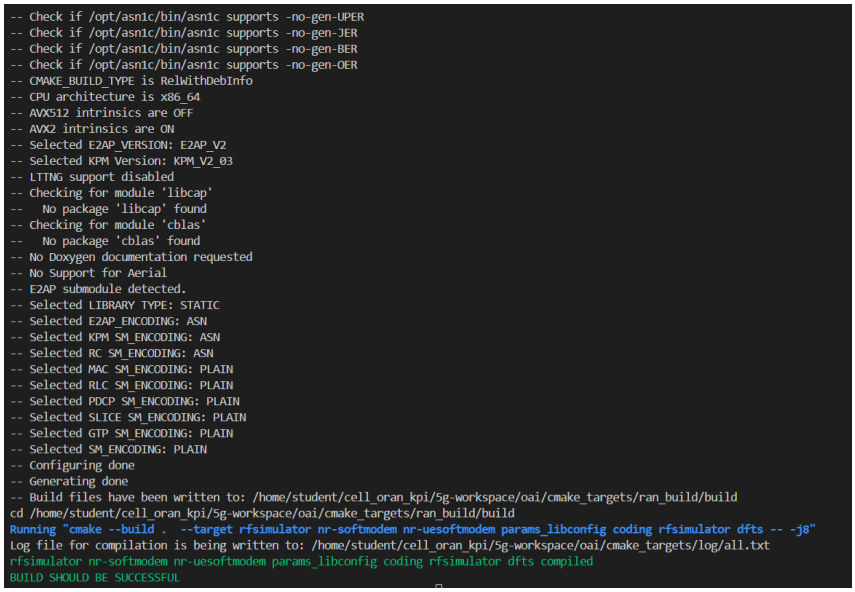

Figure 1 : Build OAI

- Build FlexRIC

```
$ Build FlexRIC

cd ~/cell_oran_kpi/5g-workspace/flexric/
mkdir build && cd build && cmake .. && make -j8
```

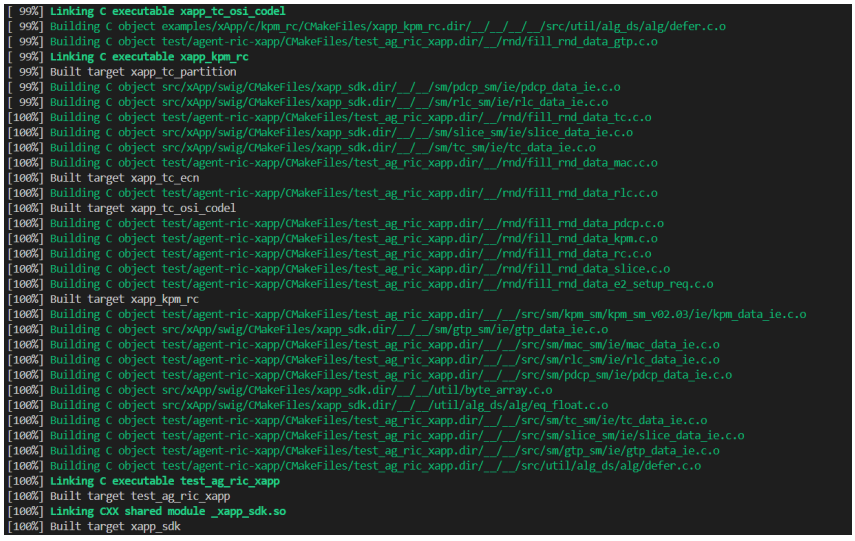

Figure 2 : Build FlexRIC

- Installation of Service Models (SMs)

```
$ Service Models

cd ~/cell_oran_kpi/5g-workspace/flexric/build
sudo make install
```

```
Install the project...
-- Install configuration: "Debug"
-- Installing: /usr/local/lib/flexric/libmac_sm.so
-- Installing: /usr/local/lib/flexric/librlc_sm.so
-- Installing: /usr/local/lib/flexric/libpdcp_sm.so
-- Installing: /usr/local/lib/flexric/libslice_sm.so
-- Installing: /usr/local/lib/flexric/libtc_sm.so
-- Installing: /usr/local/lib/flexric/libgtp_sm.so
-- Installing: /usr/local/lib/flexric/libkpm_sm.so
-- Installing: /usr/local/lib/flexric/librc_sm.so
-- Up-to-date: /usr/local/etc/flexric/flexric.conf
```

Figure 3 : Service Models Installation

- Start the 5G Core Network with `docker-compose`

```
docker compose -f
/home/student/cell_oran_kpi/5g-workspace/oai-cn5g/docker-compose.yaml up -d
```

```
student@lab-vm-5:~$ docker compose -f ~/cell_oran_kpi/5g-workspace/oai-cn5g/docker-compose.yaml up -d
WARN[0000] No services to build
[+] up 11/11
 ✓ Network oai-cn5g-public-net Created                                                                    0.1s
 ✓ Container oai-ext-dn        Created                                                                    0.2s
 ✓ Container oai-nrf           Created                                                                    0.2s
 ✓ Container ims               Created                                                                    0.2s
 ✓ Container mysql             Created                                                                    0.2s
 ✓ Container oai-udr           Created                                                                    0.1s
 ✓ Container oai-udm           Created                                                                    0.1s
 ✓ Container oai-ausf          Created                                                                    0.1s
 ✓ Container oai-amf           Created                                                                    0.1s
 ✓ Container oai-smf           Created                                                                    0.2s
 ✓ Container oai-upf           Created                                                                    0.1s
```

Figure 4 : 5g Core Network deployment

```
student@lab-vm-5:~$ docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED         STATUS                   PORTS                                                                                  NAMES
f09153b544d5   oaisoftwarealliance/oai-upf:develop            "sh /openair-upf/bin…"   22 minutes ago  Up 22 minutes (healthy)  2152/udp, 8805/udp, 5342-5344/tcp                                                      oai-upf
f29b490dc6a6   oaisoftwarealliance/oai-smf:develop            "/openair-smf/bin/oa…"   22 minutes ago  Up 22 minutes (healthy)  80/tcp, 5342-5344/tcp, 8080/tcp, 9090/tcp, 8805/udp                                    oai-smf
23b0434e5bc4   oaisoftwarealliance/oai-amf:develop            "/openair-amf/bin/oa…"   22 minutes ago  Up 22 minutes (healthy)  80/tcp, 5342-5344/tcp, 8080/tcp, 9090/tcp, 38412/sctp                                  oai-amf
a3d45a26c780   oaisoftwarealliance/oai-ausf:develop           "/openair-ausf/bin/o…"   22 minutes ago  Up 22 minutes (healthy)  80/tcp, 5342-5344/tcp, 8080/tcp                                                        oai-ausf
2a349ccf9935   oaisoftwarealliance/oai-udm:develop            "/openair-udm/bin/oa…"   22 minutes ago  Up 22 minutes (healthy)  80/tcp, 5342-5344/tcp, 8080/tcp                                                        oai-udm
6e64feca6344   oaisoftwarealliance/oai-udr:develop            "/openair-udr/bin/oa…"   22 minutes ago  Up 22 minutes (healthy)  80/tcp, 8080/tcp                                                                       oai-udr
7f26993ad53f   oaisoftwarealliance/oai-nrf:develop            "/openair-nrf/bin/oa…"   22 minutes ago  Up 22 minutes (healthy)  80/tcp, 5342-5344/tcp, 8080/tcp                                                        oai-nrf
3c78cc6ff516   oaisoftwarealliance/ims:latest                 "asterisk -fp"           22 minutes ago  Up 22 minutes (healthy)                                                                                         ims
b59298c9e290   mysql:8.0                                      "docker-entrypoint.s…"   22 minutes ago  Up 22 minutes (healthy)  3306/tcp, 33060/tcp                                                                    mysql
1c6f2a9710fa   oaisoftwarealliance/trf-gen-cn5g:jammy         "/bin/bash -c ' ip r…"   22 minutes ago  Up 22 minutes (healthy)                                                                                         oai-ext-dn
```

Figure 5 : Checking if the 5G CN is deployed

- Start the gNB in `terminal 1` :

```
$ gNB
```

```
cd ~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build
   /build
sudo ./nr-softmodem -O ../../../targets/PROJECTS/GENERIC-NR
   -5GC/CONF/gnb.sa.band78.fr1.106PRB.usrpb210.conf --gNBs
   .[0].min_rxtxtime 6 --rfsim --sa
```

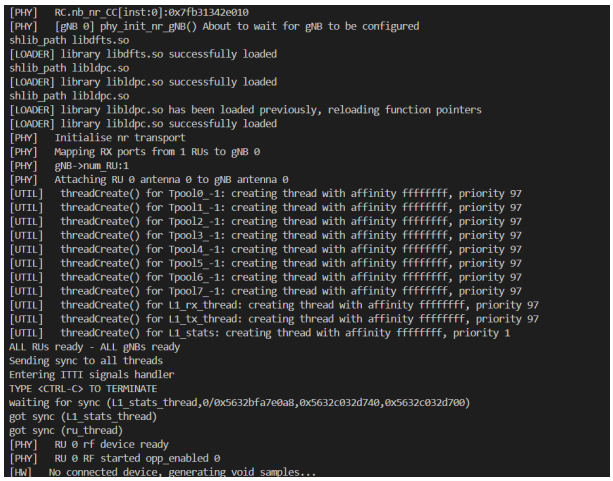

Figure 6 : gNB start

- Start the UE in `terminal 2` :

```
$ UE
```

```
cd ~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build
   /build
sudo ./nr-uesoftmodem -r 106 --numerology 1 --band 78 -C
   3619200000 --rfsim --sa --uicc0.imsi 001010000000001 --
   rfsimulator.serveraddr 127.0.0.1
```

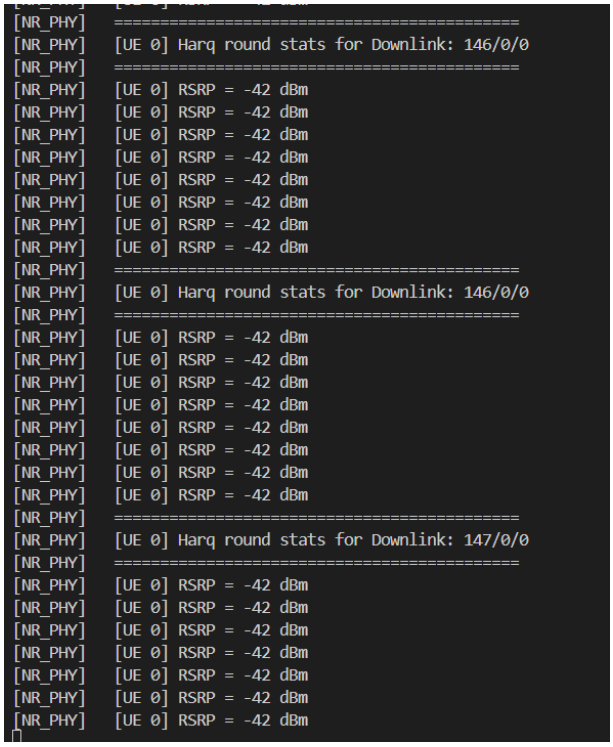

Figure 7 : Starting the UE

- Start the near-RT RIC in `terminal 3` :
  `./cell_oran_kpi/5g-workspace/flexric/build/examples/ric/nearRT-RIC`

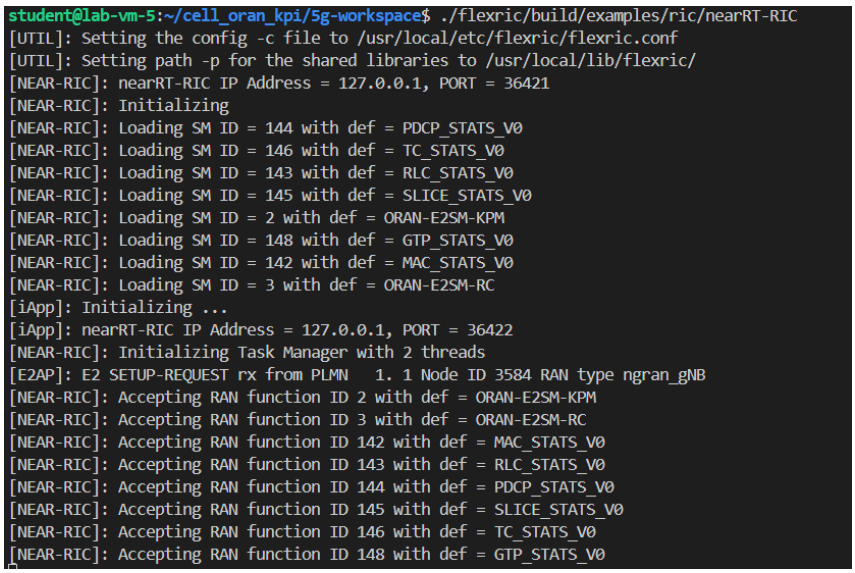

Figure 8 : Starting the near-RT RIC

- Ping between UE and network in `terminal 4` :
  The environment is successfully build as shown here
  `ping -c 3 192.168.70.135 -I oaitun_ue1`

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build$ ping -c 3 192.168.70.135 -I oaitun_ue1
PING 192.168.70.135 (192.168.70.135) from 10.0.0.2 oaitun_ue1: 56(84) bytes of data.
64 bytes from 192.168.70.135: icmp_seq=1 ttl=63 time=17.0 ms
64 bytes from 192.168.70.135: icmp_seq=2 ttl=63 time=20.2 ms
64 bytes from 192.168.70.135: icmp_seq=3 ttl=63 time=16.4 ms

--- 192.168.70.135 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 16.354/17.839/20.165/1.665 ms
```

Figure 9 : Uplink UE to Network

`sudo docker exec -it oai-ext-dn ping -c 3 10.0.0.2`

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build$ sudo docker exec -it oai-ext-dn ping -c 3 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=63 time=18.5 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=63 time=19.8 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=63 time=21.1 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 18.486/19.799/21.127/1.078 ms
```

Figure 10 : Downlink ping Network to UE

7 CPU Metric Processing and Usage

This section details the implementation of the new performance metric integrated into the O-RAN architecture to monitor computing resources.

7.1 Metric Definition and Objective

The `gNB_CPU_Load` metric was designed to measure the actual CPU load consumed by the base station (gNB) process. Unlike a system-wide measurement, this metric specifically focuses on resource usage by the OAI protocol layers (PHY, MAC, RLC, PDCP, and RRC). The primary objective is to enable the controller (RIC) to take proactive actions, such as :

- **Overload detection** : Identifying whether signal processing functions are saturating CPU cores.
- **Load balancing** : Redirecting new user equipments (UEs) to another cell if the CPU load exceeds a predefined threshold.
- **Energy optimization** : Reducing allocated resources during periods of low traffic to save energy.

7.2 Data Collection and Computation

The computation of the `gNB_CPU_Load` metric relies on the exploitation of Linux kernel data structures through the E2 agent integrated into the OpenAirInterface source code. This approach enables an accurate estimation of the CPU resources consumed specifically by the gNB process.
To provide a meaningful CPU load percentage, the agent computes a differential between two measurement intervals (t_1 and t_2). Conceptually, the CPU load corresponds to the ratio between the CPU time consumed by the gNB process and the real elapsed time between two measurements :

$$\text{CPU Load} = \frac{\Delta\text{CPU Time}}{\Delta\text{Wall\_Clock\_Time}} \times 100 \tag{1}$$

This conceptual formulation can be expressed more precisely by accounting for both user-mode execution time and kernel-mode system calls. The resulting detailed equation is given by :

$$\text{CPU Load} = \frac{(U_2 + S_2) - (U_1 + S_1)}{T_2 - T_1} \times 100 \tag{2}$$

where :

- U represents the user CPU time (`ru_utime`) ;
- S represents the system CPU time (`ru_stime`) ;
- T represents the wall-clock elapsed time between two measurement instants.

The overall process consists of three key steps :

1. **System data retrieval** : The E2 agent invokes the system call `getrusage(RUSAGE_SELF)` in the file `ran_func_kpm_subs.c` to retrieve the CPU time consumed by the gNB process.
2. **Differential computation** : The current measurement is compared with the previous one to obtain a real-time estimation of the CPU load.
3. **E2 transmission** : The computed CPU load value is encapsulated into a KPM indication message (Service Model ID 2) and transmitted to the FlexRIC controller via the E2 interface.

7.3 Interpretation of the Metric Value

It is crucial to understand that the reported value may exceed 100%.
Multi-core interpretation
In a Linux environment, a load of 100% corresponds to the full utilization of a single

CPU core. Since OpenAirInterface is a multi-threaded application leveraging multiple cores for parallel radio signal processing, a value of 157.77% indicates that the gNB is using approximately 1.6 CPU cores simultaneously.

Performance analysis

- **Low value (e.g., $< 50\%$)** : The gNB is idle or handling very few UEs.
- **Medium value (e.g., 100%–200%)** : Normal operation with active data traffic.
- **Critical value** : If the metric approaches the maximum number of available CPU cores on the host machine, processing delays (latency) may occur on the radio interface.

8 Implementation

- Path : `5g-workspace/oai/openair2/E2AP/RAN_FUNCTION/O-RAN/ran_func_kpm_subs.c`
- Add the function that calculates CPU usage using Linux system calls.

ran_func_kpm_subs.c

```c
#include <sys/time.h>
#include <sys/resource.h>

static struct rusage last_usage;
static struct timeval last_time;
static int first_run = 1;

static meas_record_list_t fill_CPU_Metric(uint32_t
   gran_period_ms, cudu_ue_info_t *ue_info, const size_t
   ue_idx) {
    meas_record_list_t meas_record = {0};
    meas_record.value = REAL_MEAS_VALUE;

    struct rusage current_usage;
    struct timeval current_time;

    getrusage(RUSAGE_SELF, &current_usage);
    gettimeofday(&current_time, NULL);

    if (first_run) {
        last_usage = current_usage;
        last_time = current_time;
        first_run = 0;
        meas_record.real_val = 0.0;
        return meas_record;
    }

    double user_delta = (double)(current_usage.ru_utime.
```

```
    tv_sec - last_usage.ru_utime.tv_sec) +
27                          (double)(current_usage.ru_utime.
    tv_usec - last_usage.ru_utime.tv_usec) / 1000000.0;
28
29      double sys_delta  = (double)(current_usage.ru_stime.
    tv_sec - last_usage.ru_stime.tv_sec) +
30                          (double)(current_usage.ru_stime.
    tv_usec - last_usage.ru_stime.tv_usec) / 1000000.0;
31
32      double time_delta = (double)(current_time.tv_sec -
    last_time.tv_sec) +
33                          (double)(current_time.tv_usec -
    last_time.tv_usec) / 1000000.0;
34
35      if (time_delta > 0) {
36          meas_record.real_val = ((user_delta + sys_delta) /
    time_delta) * 100.0;
37      } else {
38          meas_record.real_val = 0.0;
39      }
40
41      last_usage = current_usage;
42      last_time = current_time;
43
44      return meas_record;
45  }
```

- Path : `5g-workspace/oai/openair2/E2AP/RAN_FUNCTION/O-RAN/ran_func_kpm_subs.c`
- Register the function in the `lst_measure` array and specify which function the metric will use.

ran_func_kpm_subs.c

```
1  static kv_measure_t lst_measure[] = {
2      {.key = "gNB_CPU_Load", .value = fill_CPU_Metric }, //
     registration of our metric
3      ...
4  };
```

- Path : `5g-workspace/oai/openair2/E2AP/RAN_FUNCTION/O-RAN/ran_func_kpm.c`
- Add the metric name string so the E2 Agent advertises it.

ran_func_kpm.c

```
1  static const char* kpm_meas_gnb[] = {
2    "DRB.PdcpSduVolumeDL",
```

```
    "DRB.PdcpSduVolumeUL",
    "DRB.RlcSduDelayDl",
    "DRB.UEThpDl",
    "DRB.UEThpUl",
    "RRU.PrbTotDl",
    "RRU.PrbTotUl",
    "gNB_CPU_Load", // our metric name for E2 Agent to
     advertises
    NULL,
};
```

- Path : `5g-workspace/flexric/examples/xApp/c/monitor/xapp_kpm_moni.c`
- Update the xApp by adding a print statement to the logging function for our metric.

xapp_kpm_moni.c

```
static void log_real_value(byte_array_t name,
    meas_record_lst_t meas_record){
   ...
   } else if (cmp_str_ba("DRB.UEThpUl", name) == 0) {
     printf("DRB.UEThpUl = %.2f [kbps]\n", meas_record.
    real_val);
   } else if (cmp_str_ba("gNB_CPU_Load", name) == 0){
     printf("gNB CPU Load = %.2f %%\n", meas_record.real_val);
   } else {
     printf("Measurement Name not yet supported\n");
   }
```

At this point, we must rebuild the OAI softmodem to include the new C logic in the gNB and recompile the xApp by following the same steps as described in Section 6.

9 Results

The following results demonstrate the correct reporting and consumption of the proposed CPU usage KPM, illustrated through xApp outputs and concurrent traffic measurements using `iperf`.

9.1 Experimental Observations

The following tests involve one client and one server within the network :

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace/flexric$ ./build/examples/xApp/c/monitor/xapp_kpm_moni
[UTIL]: Setting the config -c file to /usr/local/etc/flexric/flexric.conf
[UTIL]: Setting path -p for the shared libraries to /usr/local/lib/flexric/
[xAapp]: Initializing ...
[xApp]: nearRT-RIC IP Address = 127.0.0.1, PORT = 36422
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libpdcp_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libtc_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/librlc_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libslice_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libkpm_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libgtp_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libmac_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/librc_sm.so
[NEAR-RIC]: Loading SM ID = 144 with def = PDCP_STATS_V0
[NEAR-RIC]: Loading SM ID = 146 with def = TC_STATS_V0
[NEAR-RIC]: Loading SM ID = 143 with def = RLC_STATS_V0
[NEAR-RIC]: Loading SM ID = 145 with def = SLICE_STATS_V0
[NEAR-RIC]: Loading SM ID = 2 with def = ORAN-E2SM-KPM
[NEAR-RIC]: Loading SM ID = 148 with def = GTP_STATS_V0
[NEAR-RIC]: Loading SM ID = 142 with def = MAC_STATS_V0
[NEAR-RIC]: Loading SM ID = 3 with def = ORAN-E2SM-RC
[xApp]: DB filename = /tmp/xapp_db_1767809773657060
 [xApp]: E42 SETUP-REQUEST tx
[xApp]: E42 SETUP-RESPONSE rx
[xApp]: xApp ID = 7
[xApp]: Registered E2 Nodes = 1
Connected E2 nodes = 1
[xApp]: E42 RIC SUBSCRIPTION REQUEST tx RAN_FUNC_ID 2 RIC_REQ_ID 1
[xApp]: SUBSCRIPTION RESPONSE rx
[xApp]: Successfully subscribed to RAN_FUNC_ID 2
```

Figure 11 : Successful subscription to metrics

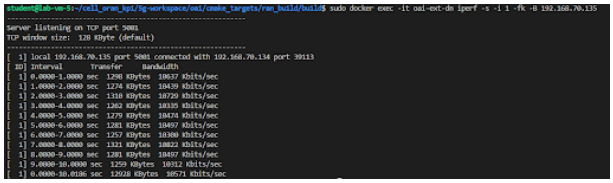

Figure 13 : Test iperf server side

```
student@lab-vm-5:~$ iperf -c 192.168.70.135 -i 1 -b 10M -B 10.0.0.2
------------------------------------------------------------
Client connecting to 192.168.70.135, TCP port 5001
TCP window size: 85.0 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.2 port 39113 connected with 192.168.70.135 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-1.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 1.0000-2.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 2.0000-3.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 3.0000-4.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 4.0000-5.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 5.0000-6.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 6.0000-7.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 7.0000-8.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 8.0000-9.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 9.0000-10.0000 sec  1.25 MBytes  10.5 Mbits/sec
[  1] 0.0000-10.0550 sec  12.6 MBytes  10.5 Mbits/sec
```

Figure 14 : Test iperf client side

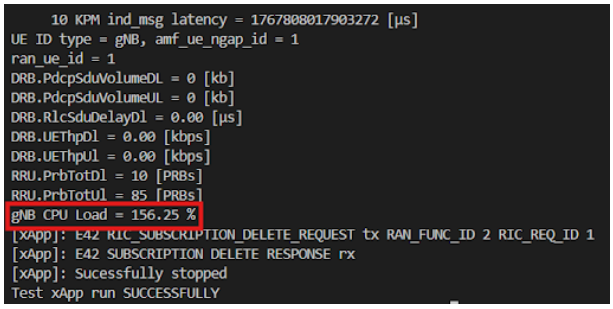

Figure 12 : CPU Usage xApp Result 2

The following tests involve 5 clients and one server within the network :

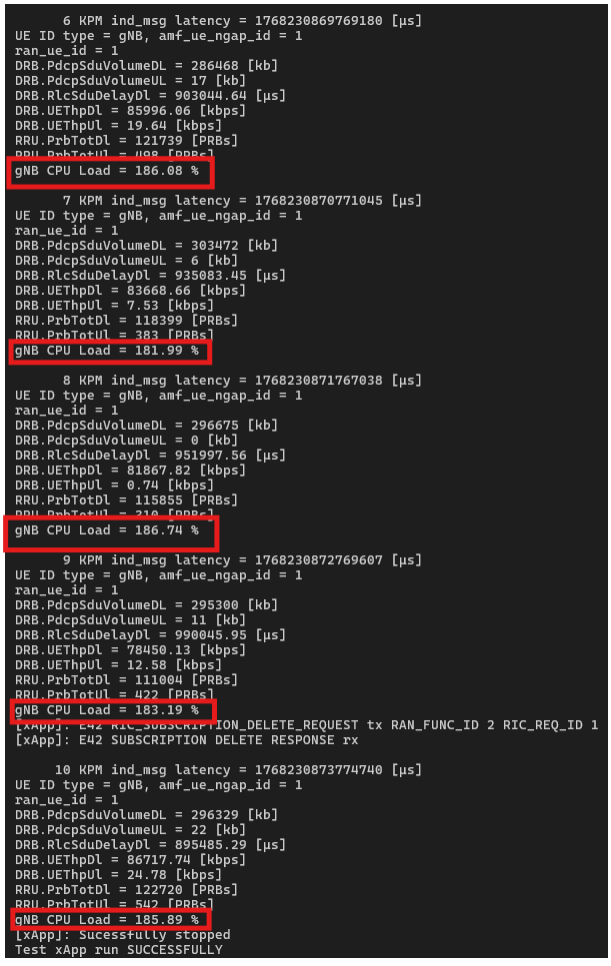

Figure 15 : xApp KPM result for he load test

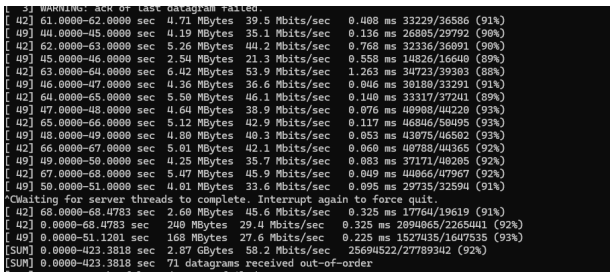

Figure 16 : server side for the `iperf` test

9.2 Analysis

The implementation and testing of the `gNB_CPU_Load` KPI demonstrate a successful extension of the O-RAN KPM Service Model, providing critical visibility into the gNB's computational state. By comparing the results across different user loads, we can analyze how the software-based RAN scales under pressure and identify the primary drivers of CPU consumption.

Table 1 : Comparison of gNB CPU Load : Single vs. Multiple Clients

Scenario	**CPU Load (%)**	**Technical Observation**
No Traffic (Idle)	$\approx$ 150% – 155%	Baseline for PHY sync and busy-waiting.
1 Client (iperf)	156.25%	Minimal overhead for single-user data.
5 Clients (iperf)	181.99% – 186.74%	Scaling for RRC context and scheduling.

The data reveals that the transition from a single client to five concurrent clients results in a relatively small increase in CPU utilization (approximately 18% to 30% additional load). In our Linux-based environment, where 100% represents the saturation of one core, the baseline of 156.25% confirms that OpenAirInterface is a multi-threaded application utilizing roughly 1.6 cores. The fact that the load does not quintuple with five times the users indicates that the system is computationally efficient at scaling ; the additional processing is largely dedicated to managing more Radio Resource Control (RRC) contexts and the increased complexity of the MAC layer scheduler.

A particularly noteworthy observation is that the CPU load remains high (above 150%) even when no active iperf traffic is present. This "High Idle Load" is a hallmark of software-defined radio (SDR) platforms like OpenAirInterface. To maintain the 5G cell, the physical layer (PHY) must continuously process radio frames, broadcast system information, and listen for incoming signals from UEs every millisecond. Furthermore, OAI often utilizes "busy-waiting" loops to minimize processing latency, which keeps the CPU cores in a high-power state to respond instantly to radio events. Thus, the computational budget is dominated by a fixed "maintenance cost" of the radio stack, while user-specific traffic

represents a variable but smaller cost.

In summary, the `gNB_CPU_Load` metric provides an accurate and reactive measurement of the gNB's processing health. The results confirm that while the gNB scales well with additional users, the underlying SDR architecture requires a significant baseline of computational resources to maintain network synchronization and real-time performance.